

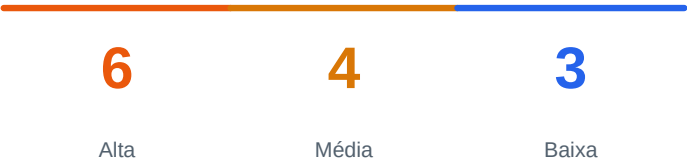
RELATÓRIO DE AUDITORIA DE SEGURANÇA ZERO-TRUST

Lúmen — rodada 2

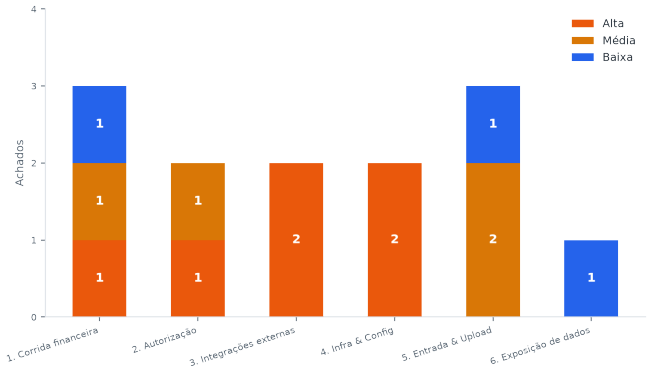
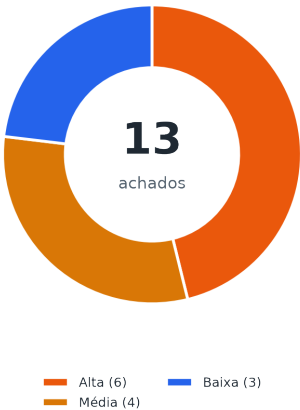
Framework de 15 leis de arquitetura segura, aplicado camada por camada — perímetro, identidade/autorização, lógica de negócio/dados e infraestrutura/supply chain. Metodologia: 6 agentes de auditoria independentes, cada um cobrindo uma camada, achados cruzados e consolidados manualmente.

Repositório	rodartepradoadvogados/lumen
Data desta auditoria	05 de setembro de 2026
Auditoria anterior	01 de setembro de 2026 — 5 achados (F1-F5), todos corrigidos e sem regressão (verificado nesta rodada)
Escopo	Autenticação/multi-tenant, Server Actions, integrações externas (SSRF/OAuth/webhooks), segredos/config/headers, código novo da sessão, corridas em fluxo financeiro
Status	NENHUMA correção deste relatório foi aplicada ao código — Fase 2 e 3 são propostas aguardando validação

Resumo executivo



Achados por severidade e por categoria



13 achados confirmados nesta rodada, nenhum crítico. Zero regressão nos 5 achados da auditoria de 01/09/2026. Maior risco concentrado no módulo financeiro (corrida de pagamento duplicado, V1) e numa integração adicionada depois do último endurecimento de segurança (BTG, V3) — ambos os padrões de correção JÁ existem em outro lugar do próprio código, só não foram replicados nesses dois pontos.

Scorecard de segurança

Vulnerabilidades CRÍTICAS	0
Vulnerabilidades ALTAS	6
Vulnerabilidades MÉDIAS	4
Vulnerabilidades BAIXAS	3
Anti-padrões de vibe coding detectados	A5 parcial (BTG sem o nonce que as outras 3 integrações OAuth já usam) · nenhum outro dos A1-A10 confirmado
Nota geral	C — funcional e com isolamento de tenant sólido, mas com gaps reais que precisam virar prioridade de sprint antes de crescer mais o produto

Top 3 ações prioritárias

- 1. Corrigir a corrida de pagamento duplicado no financeiro (V1) — trava de linha + rejeição de pagamento acima do saldo.

- 2. Aplicar o mesmo `verifyAndConsumeOAuthState` do Google/Microsoft/Dropbox ao fluxo BTG (V3), e trocar a dependência `xlsx` (V4).
- 3. Configurar os headers de segurança ausentes (V5) e fechar a autenticação da rota `migrate-legacy` (V6).

Pontos fortes confirmados

- **1. Isolamento de tenant** — Confirmado sistemático e consistente em ~90 arquivos revisados (`lib/actions/*.ts` + `app/api/**/route.ts`): toda mutação/consulta por id valida `officeId` via `findFirst/where` antes de agir, e os helpers `isXInOffice` (`lib/officeScope.ts`) são usados antes de aceitar qualquer FK secundária vinda do cliente.
- **2. Impersonação ("atuar como")** — Regressão do achado A33 checada e PASSOU: exige `isPlatformOwner` na identidade REAL (`ignoreActing:true`), a sessão de suporte é revalidada contra o banco a cada requisição (não confia só no cookie), e `isAdmin` é zerado durante a atuação — quem está "atuando como" o escritório-cliente nunca herda privilégio de admin dele.
- **3. Fail-closed em webhooks/cron** — Todas as 13 rotas de cron, o webhook do Asaas e o webhook do WhatsApp (achado F2 da auditoria anterior) seguem o padrão correto: recusam quando o segredo de verificação está ausente. O NOVO endpoint `/api/blog/draft` (robô de conteúdo) também nasceu seguindo esse padrão — evidência de que a disciplina documentada em `CLAUDE.md` está sendo aplicada em código novo.
- **4. Criptografia/hashing** — JWT via ``jose`` com verificação de assinatura de verdade (HS256), nunca decode sem `verify`; `bcrypt` com custo 10 em todos os pontos de hash de senha; SSRF do achado A63 (upload por URL) permanece fechado — allowlist estrita a `*.blob.vercel-storage.com` em todo call site atual.
- **5. Auditoria anterior (01/09/2026)** — Os 5 achados da primeira rodada (F1-F5) seguem corrigidos sem regressão: webhook WhatsApp fail-closed, protocolo javascript: bloqueado em `meetingUrl/tribunalLink`, `seed.ts` sem senha real fixa, e-mails HTML escapados, foto de perfil exigindo sessão.

Fase 1 — Visão do atacante (Red Team)

Cada achado abaixo foi lido diretamente no código atual do repositório antes de entrar neste relatório — arquivo, linha e trecho reais.

Sev.	Categoria / Arquivo	Descrição
Alta	1. Corrida financeira lib/actions/financeiro.ts:282-419 (markPayablePaid, markReceivablePaid, markManyPayablesPaid, m	V1 — Dar baixa (individual e em bloco) sem trava de corrida — pagamento pode ser contado em dobro
Alta	2. Autorização lib/actions/cases.ts:288-382 (updateCase)	V2 — updateCase permite que qualquer usuário do escritório altere os valores-base do honorário percentual, sem gate financeiro nem auditoria
Alta	3. Integrações externas app/api/btg/callback/route.ts, app/api/btg/connect/route.ts, lib/btg.ts:53-90	V3 — Conexão OAuth com o BTG Empresas sem verificação de state — CSRF / injeção de código de autorização
Alta	3. Integrações externas package.json:35 ("xlsx": "^0.18.5"), lib/importers/parse.ts:5-9	V4 — Dependência xlsx (SheetJS) travada numa versão com Prototype Pollution e ReDoS conhecidos, sem correção disponível via npm
Alta	4. Infra & Config next.config.mjs, middleware.ts, vercel.json	V5 — Nenhum header de segurança configurado (CSP, X-Frame-Options, HSTS, X-Content-Type-Options)
Alta	4. Infra & Config app/api/admin/migrate-legacy/route.ts	V6 — Rota de migração sem verificação de sessão — só um segredo em query string, com erro cru devolvido ao cliente
Média	1. Corrida financeira lib/actions/tasks.ts:182-224 (delegateTask), lib/actions/publications.ts:196-244 (submitPublica	V7 — Distribuição/delegação de publicação sem compare-and-swap — a mesma publicação pode ser atribuída duas vezes
Média	5. Entrada & Upload app/(app)/processos/page.tsx:106-118, app/(app)/contatos/clientes/page.tsx:15-19, lib/actions/s	V8 — Sem paginação/limite em listas centrais e na busca global — leitura completa do tenant a cada render/tecla
Média	5. Entrada & Upload lib/richText.ts:19-33, components/a notacoes/AnotacoesPessoaisList.tsx:64	V9 — Sanitizador de HTML artesanal (regex, sem parser real) alimentando dangerouslySetInnerHTML
Média	2. Autorização lib/actions/financeiro.ts:258,1084, 1222 (syncReceivableStatus, ensureRecurringFeeReceivables, e	V10 — Server Actions aceitam offceld como parâmetro cru em vez de derivar da sessão (violação estrutural da Lei 5)
Baixa	1. Corrida financeira lib/actions/apuracao.ts:73-166 (apurarHonorario)	V11 — apurarHonorario não-transacional (corrida possível, mas sem duplicar dinheiro — só uma tarefa de lembrete)

Sev.	Categoria / Arquivo	Descrição
Baixa	5. Entrada & Upload app/api/photos/upload/route.ts:28, app/api/perfil/foto/upload/route.ts:24	V12 — Upload de foto confia só no MIME declarado pelo navegador — aceita SVG com script embutido
Baixa	6. Exposição de dados app/api/photos/file/[id]/route.ts	V13 — Rota pública de biblioteca de fotos sem checagem de sessão/officeId — vazamento cross-tenant de imagens decorativas

Evidência e exploração por achado

Alta

V1 — Dar baixa (individual e em bloco) sem trava de corrida — pagamento pode ser contado em dobro

```
lib/actions/financeiro.ts:282-419 (markPayablePaid, markReceivablePaid, markManyPayablesPaid, markManyReceivablesPaid) + lib/actions/assessoria.ts:550-563 (markHonorarioPaid)
```

```
export async function markPayablePaid(id: string, paidAmount: number, paidDate: string, ...) {
  const officeId = await requireFinanceOfficeId();
  const existing = await prisma.payable.findFirst({ where: { id, officeId }, select: { id: true, caseId: true } });
  if (!existing) throw new Error("Conta a pagar não encontrada.");
  // nenhuma checagem de status/saldo aqui
  await prisma.financePayment.create({ data: { officeId, amount: paidAmount, ..., payableId: id } });
  await syncPayableStatus(id, officeId);
  ...
}
```

Exploração: Duas requisições markPayablePaid(id, 5000, ...) disparadas com poucos milissegundos de diferença (duas abas abertas na mesma conta, ou um clique duplo que o React ainda não desabilitou) passam as duas pelo findFirst antes de qualquer uma escrever. Resultado: dois FinancePayment de R\$5000 no lugar de um — a conta aparece paga com R\$10000, e esse valor duplicado entra direto no Livro Caixa, DRE e Fluxo de Caixa (que somam FinancePayment, não o campo legado paidAmount). O mesmo vale para markManyPayablesPaid/markManyReceivablesPaid (o "Dar baixa em bloco"): o saldo de cada item é calculado uma vez, antes do loop, então duas chamadas do lote inteiro se sobrepondo duplicam a baixa de cada conta selecionada.

Condições: Nenhuma condição especial — duplo clique, duas abas abertas na mesma conta, ou uma requisição HTTP repetida (retry de rede) já é suficiente. Não exige nenhum privilégio além do acesso normal ao Financeiro.

Alta**V2 — updateCase permite que qualquer usuário do escritório altere os valores-base do honorário percentual, sem gate financeiro nem auditoria**

lib/actions/cases.ts:288-382 (updateCase)

```
export async function updateCase(caseId: string, data: { ...; caseValue?: string; convictionValue?: string; economicBenefitValue?: string; ... }) {
  const viewer = await getCurrentUser(); // só verifica login, não financeAccess
  if (!viewer) return { error: "Sessão inválida." };
  const existing = await prisma.case.findFirst({ where: { id: caseId, officeId: viewer.officeId }, ... });
  ...
  await prisma.case.update({ where: { id: caseId }, data: {
    caseValue: data.caseValue ? parseFloat(data.caseValue) : null,
    convictionValue: data.convictionValue ? parseFloat(data.convictionValue) : null,
    economicBenefitValue: data.economicBenefitValue ? parseFloat(data.economicBenefitValue) : null,
    ...
  }});
}
```

Exploração: Um estagiário ou advogado sem financeAccess, com permissão normal de editar o cadastro do processo, define convictionValue: "999999" num caso que tem (ou terá) honorário sucumbencial percentual. Quando o financeiro rodar apurarHonorario/createHonorarioLancamento, o valor cobrado do cliente é calculado sobre um número que essa pessoa inventou — sem que financeAccess/apuradoPorId tenham entrado em nenhum momento na jogada.

Condições: Qualquer usuário ativo do escritório com acesso normal à edição de Processo (não precisa de financeAccess nem isAdmin).

Alta**V3 — Conexão OAuth com o BTG Empresas sem verificação de state — CSRF / injeção de código de autorização**

app/api/btg/callback/route.ts, app/api/btg/connect/route.ts, lib/btg.ts:53-90

```
// lib/btg.ts
export function getBtgAuthorizeUrl(state: string): string {
  const params = new URLSearchParams({ ..., state }); // state = user.id, não um nonce
}

// app/api/btg/connect/route.ts
return NextResponse.redirect(getBtgAuthorizeUrl(user.id));

// app/api/btg/callback/route.ts — nunca lê searchParams.get("state")
const code = request.nextUrl.searchParams.get("code");
const result = await exchangeBtgCode(code);

// exchangeBtgCode grava numa conexão ÚNICA para toda a plataforma:
await prisma.btgConnection.deleteMany({});
await prisma.btgConnection.create({ data: { accessToken, refreshToken, expiresAt } });
```

Exploração: O atacante inicia o fluxo OAuth com a PRÓPRIA conta BTG (id.btgactual.com, usando o client_id público do app registrado), captura o code que o BTG devolve para o navegador dele, e envia esse link para o platform owner (já logado): https://<lumen>/api/btg/callback?code=<code do atacante>. Como o callback já confirma isPlatformOwner mas não confirma que o code pertence à MESMA sessão que iniciou o fluxo (sem state, sem nonce), ao clicar o admin faz o servidor trocar o code do atacante e sobrescrever a única conexão BTG da plataforma pela conta do atacante — a plataforma passaria a emitir boletos (e ler dados de boleto, escopo bank-slips.readonly) contra a conta bancária do atacante em vez da do escritório.

Condições: Exige que o platform owner esteja logado e clique no link malicioso (engenharia social/phishing de um único clique) — clássico "login CSRF"/OAuth code injection.

Alta**V4 — Dependência xlsx (SheetJS) travada numa versão com Prototype Pollution e ReDoS conhecidos, sem correção disponível via npm**`package.json:35 ("xlsx": "^0.18.5"), lib/importers/parse.ts:5-9`

```
// package.json
"xlsx": "^0.18.5",

// lib/importers/parse.ts
export async function parseSpreadsheet(file: File): Promise<Row[]> {
  const buffer = await file.arrayBuffer();
  const wb = XLSX.read(buffer, { type: "array", cellDates: false }); // conteúdo não confiável
  ...
}
```

Exploração: Alcançável por `importCases/importFinance/importAgenda` (`lib/actions/import.ts`), gated por `isAdmin/financeAccess` — ou seja, exige uma conta admin/financeiro comprometida ou um insider malicioso, não um estranho. Essa conta faz upload de uma planilha `.xlsx/.xls` malformada especificamente construída para disparar o ReDoS conhecido (trava o processo Node servindo TODOS os escritórios da plataforma) ou o Prototype Pollution.

Condições: Requer uma conta com `isAdmin` ou `financeAccess` (phishing, credencial vazada, ou insider).

Alta**V5 — Nenhum header de segurança configurado (CSP, X-Frame-Options, HSTS, X-Content-Type-Options)**`next.config.mjs, middleware.ts, vercel.json`

```
// next.config.mjs
/** @type {import('next').NextConfig} */
const nextConfig = {};
export default nextConfig;
```

Exploração: Sem `X-Frame-Options/frame-ancestors`, o site pode ser embutido num `<iframe>` de um domínio malicioso para clickjacking (ex.: sobrepor um botão invisível de "aprovar" sobre um elemento da UI real). Sem CSP, qualquer XSS que eventualmente escape das defesas pontuais existentes (V9) tem via livre para executar/exfiltrar sem nenhuma camada adicional de contenção — exatamente o cenário que "defesa em profundidade" existe para cobrir.

Condições: Nenhuma — é uma lacuna de configuração, não depende de nenhuma ação de terceiro.

Alta**V6 — Rota de migração sem verificação de sessão — só um segredo em query string, com erro cru devolvido ao cliente**`app/api/admin/migrate-legacy/route.ts`

```
export async function GET(request: NextRequest) {
  const secret = request.nextUrl.searchParams.get("secret"); // segredo na URL
  const expected = process.env.MIGRATION_SECRET;
  if (!expected || secret !== expected) return NextResponse.json({ error: "unauthorized" }, { status: 401 });
  // nenhum getCurrentUser()/isAdmin aqui
  ...
} catch (error) {
  return NextResponse.json({ error: error instanceof Error ? error.message : "...", { status: 500
}});
}
```

Exploração: Um segredo em query string fica em logs de acesso do servidor/proxy, no histórico do navegador de quem a chamou, e em qualquer header `Referer` enviado por engano — mais fácil de vazar que um `Authorization` header. Quem obtiver esse único valor tem acesso completo à rota, sem precisar de login algum no Lúmen (nenhuma conta, nenhuma sessão) — diferente de toda outra rota administrativa do projeto, que sempre soma um segundo fator (sessão + `isAdmin/isPlatformOwner`).

Condições: Requer conhecer o valor de `MIGRATION_SECRET` (vazamento de log/histórico) — mas, ao contrário das outras rotas admin, não exige NENHUMA conta/sessão além disso.

Média

V7 — Distribuição/delegação de publicação sem compare-and-swap — a mesma publicação pode ser atribuída duas vezes

lib/actions/tasks.ts:182-224
(submitPublicationDistribution)

(delegateTask),

lib/actions/publications.ts:196-244

```
for (const responsibleId of responsibleIds) {
  const task = await prisma.task.create({ data: { ..., publicationId: data.publicationId } });
  ...
}
if (data.publicationId) {
  await prisma.publication.updateMany({ where: { id: data.publicationId, officeId: viewer.officeId },
data: { assignedToId: responsibleIds[0] } }); // sem checar estado atual
}
```

Exploração: Dois advogados (ou um admin usando "Distribuir pendentes" e um colega clicando "Delegar" na mesma publicação) veem a mesma publicação como não atribuída e submetem a delegação quase ao mesmo tempo. O servidor cria DUAS Tasks (dois compromissos/prazos, duas notificações TAREFA_DELEGADA) para a mesma publicação, e o último updateMany a rodar vence silenciosamente — o outro responsável continua com uma tarefa ativa para algo que a tela mostra como "atribuído a outra pessoa".

Condições: Dois usuários agindo sobre a mesma publicação pendente em uma janela de poucos segundos — plausível numa distribuição em lote feita por um admin coincidindo com uma delegação individual.

Média

V8 — Sem paginação/limite em listas centrais e na busca global — leitura completa do tenant a cada render/tecla

app/(app)/processos/page.tsx:106-118,
lib/actions/search.ts:69-81

app/(app)/contatos/clientes/page.tsx:15-19,

```
// app/(app)/processos/page.tsx
prisma.case.findMany({ where, include: { client: true, clients: {...}, parties: true, responsible:
true, _count: {...} }, orderBy: SORTS[sortKey] }) // sem take

// lib/actions/search.ts – 4x por chamada, sem take
const caseCandidates = await prisma.case.findMany({ where: { officeId }, select: {...} });
```

Exploração: Não há vazamento cross-tenant (tudo já filtra por officeId) — é um risco de exaustão de recursos. Um escritório com histórico grande de processos/clientes paga uma leitura cada vez mais cara a cada carregamento de página ou a cada tecla no campo de busca; nada impede que uma sessão autenticada dispare essas chamadas repetidamente (não há rate-limit nas Server Actions), degradando o pool de conexão do banco (Neon) para outros tenants do mesmo plano compartilhado.

Condições: Nenhuma condição além de um histórico de dados crescente ou uma sessão autenticada disparando as chamadas repetidamente.

Média

V9 — Sanitizador de HTML artesanal (regex, sem parser real) alimentando dangerouslySetInnerHTML

lib/richText.ts:19-33, components/anotacoes/AnotacoesPessoaisList.tsx:64

```
const ALLOWED_TAGS = new Set(["b", "strong", "i", "em", "u", "ul", "ol", "li", "br", "p", "div", "span"]);
export function sanitizeRichTextHtml(html: string): string {
  let out = html;
  out =
out.replace(/<(script|style|iframe|object|embed|link|meta|form|svg)\b[>]*>[\s\S]*?<\/\1\s*>/gi, "");
  out = out.replace(/<(script|style|iframe|object|embed|link|meta|form|svg)\b[>]*<\/?>/gi, "");
  out = out.replace(/<\/?([a-zA-Z][a-zA-Z0-9]*)\b[>]*>/g, (match, tagName) => {
    const tag = tagName.toLowerCase();
    if (!ALLOWED_TAGS.has(tag)) return "";
    return match.startsWith("<\/") ? `<\/${tag}>` : `<${tag}>`;
  });
  return out.trim();
}
```

Exploração: Nenhum bypass funcional confirmado nesta rodada (testado contra , <svg onload>, tag-splitting, atributos com aspas confusas). O risco é estrutural: a superfície coberta por este sanitizador cresceu de "só Anotações" para também Descrição de Tarefa/Evento/Prazo (agosto/2026), e regex sobre HTML não cobre com segurança os casos de borda que um parser real (com uma árvore DOM de verdade) cobre — noscript/template/math, entidades malformadas, etc.

Condições: Nenhuma condição para o risco estrutural; um bypass concreto exigiria descobrir um caso de borda específico do parser HTML do navegador que a regex não replica.

Média

V10 — Server Actions aceitam officeId como parâmetro cru em vez de derivar da sessão (violação estrutural da Lei 5)

```
lib/actions/financeiro.ts:258,1084,1222 (syncReceivableStatus, ensureRecurringFeeReceivables,
ensureRecurringExpensePayables), lib/actions/attendancePendencias.ts:115
(autoResolvePendenciasForAttachment)
```

```
export async function syncReceivableStatus(id: string, officeId: string) { ... } // officeId cru
export async function ensureRecurringFeeReceivables(officeId?: string) { ... } // sem officeId = TODOS
os escritórios
export async function autoResolvePendenciasForAttachment(attendanceId: string, docType: string,
officeId: string) { ... }
```

Exploração: Não há hoje um caminho client-reachable comprovado que passe um officeId arbitrário — mas como são exports de um arquivo "use server", o Next.js as registra no manifesto de actions independentemente de qual componente as usa hoje. Uma chamada direta ao endpoint codificado da action (bypassando a UI, via devtools/fetch) com um officeId de outro tenant, ou sem argumento nenhum em ensureRecurringFeeReceivables, executaria a função mesmo assim — o gate de autorização que deveria existir simplesmente não está lá.

Condições: Requer que um atacante descubra/replique o identificador de action codificado pelo Next.js build — não é trivial, mas não é impossível, e não deveria ser a única barreira.

Baixa

V11 — apurarHonorario não-transacional (corrida possível, mas sem duplicar dinheiro — só uma tarefa de lembrete)

lib/actions/apuracao.ts:73-166 (apurarHonorario)

```
const pendentes = await prisma.receivable.findMany({ where: { status: "A_APURAR", ... } });
for (const r of pendentes) {
  await prisma.receivable.update({ where: { id: r.id }, data: { status: "PENDENTE", amount:
valorApurado, ... } });
}
await prisma.task.create({ data: { title: "Conferir trânsito em julgado", ... } }); // pode duplicar
```

Exploração: Duplo clique/duas abas no mesmo processo+base gera duas tarefas idênticas de lembrete — ruído operacional, sem impacto financeiro.

Condições: Duas submissões da mesma apuração dentro da mesma janela de milissegundos.

Baixa**V12 — Upload de foto confia só no MIME declarado pelo navegador — aceita SVG com script embutido**

app/api/photos/upload/route.ts:28, app/api/perfil/foto/upload/route.ts:24

```
if (!file.type.startsWith("image/")) { return NextResponse.json({ error: "O arquivo precisa ser uma imagem." }, { status: 400 }); }
const blob = await put(`perfil/${user.id}-${Date.now()}-${file.name}`, file, { access: "public" });
```

Exploração: Um arquivo .svg contendo <svg onload="..."> ou <script> passa pela checagem (basta declarar Content-Type: image/svg+xml) e é armazenado com acesso público. Se essa URL for aberta como navegação de topo (não só usada dentro de , que não executa SVG ativo), o script roda no domínio do Vercel Blob — separado do domínio/cookies do Lúmen, então o impacto prático é baixo (phishing/desfiguração visual, não roubo de sessão), mas seria trivial de fechar.

Condições: Qualquer usuário autenticado (perfil/foto/upload é autoatendimento, sem exigir admin).

Baixa**V13 — Rota pública de biblioteca de fotos sem checagem de sessão/officeId — vazamento cross-tenant de imagens decorativas**

app/api/photos/file/[id]/route.ts

```
export async function GET(_request: Request, { params }: { params: { id: string } }) {
  const photo = await prisma.photo.findUnique({ where: { id: params.id } }); // sem officeId
  ...
}
```

Exploração: Quem enumerar/adivinhar um id de Photo (cuid) vê a imagem de QUALQUER escritório-cliente da plataforma, publicada ou não. Sem PII de cliente — só fotografia decorativa/institucional.

Condições: Conhecer ou adivinhar um id de Photo válido; nenhuma sessão necessária.

Fase 2 — Código blindado (Blue Team)

Nada abaixo foi aplicado ao repositório. São propostas de correção — apenas o trecho vulnerável de cada arquivo é reescrito, com comentários inline explicando o porquê de cada trava (padrão [SEGURANÇA] [id]). Aguardando validação antes de virar commit.

V1 — Serializar a baixa por linha com um lock explícito (SELECT ... FOR UPDATE) dentro de uma transação, recalculando o saldo já dentro do lock, e recusar uma baixa que ultrapasse o saldo em aberto — assim a segunda chamada concorrente vê o estado já atualizado pela primeira e é rejeitada em vez de duplicar.

```
// financeiro.ts - markPayablePaid corrigido (mesmo padrão para markReceivablePaid).
// [SEGURANÇA] [V1]: lock de linha + saldo recalculado dentro da transação - impede que
// duas chamadas concorrentes leiam o mesmo saldo "em aberto" e criem dois pagamentos.
export async function markPayablePaid(id: string, paidAmount: number, paidDate: string, receiptNumber?:
string, paymentMethod?: string, bankAccountId?: string) {
  const officeId = await requireFinanceOfficeId();
  if (bankAccountId) await assertFinanceRelationsInOffice({ bankAccountId }, officeId);

  await prisma.$transaction(async (tx) => {
    // [SEGURANÇA] Lock pessimista: nenhuma outra transação lê/escreve esta linha até esta commitar.
    const locked = await tx.$queryRaw<{ id: string; caseId: string | null; amount: number; discount:
number; surcharge: number }[]>`
      SELECT id, "caseId", amount, discount, surcharge FROM "Payable"
      WHERE id = ${id} AND "officeId" = ${officeId}
      FOR UPDATE
    `;
    const payable = locked[0];
    if (!payable) throw new Error("Conta a pagar não encontrada.");

    const pagos = await tx.financePayment.aggregate({ where: { payableId: id }, _sum: { amount: true }
});
    const soma = pagos._sum.amount ?? 0;
    const saldo = valorLiquido(payable.amount, payable.discount, payable.surcharge) - soma;
    // [SEGURANÇA] [V1]: uma segunda chamada concorrente (ou um retry) já vê saldo <= 0
    // aqui, porque a primeira já commitou dentro do lock - recusa em vez de duplicar.
    if (saldo <= 0.005) throw new Error("Este lançamento já foi quitado (baixa duplicada recusada).");
    if (paidAmount > saldo + 0.005) throw new Error(`Valor informado (${paidAmount}) é maior que o
saldo em aberto (${saldo.toFixed(2)}).`);

    await tx.financePayment.create({
      data: { officeId, amount: paidAmount, paidDate: new Date(paidDate), paymentMethod: paymentMethod
|| null, documentNumber: receiptNumber || null, bankAccountId: bankAccountId || null, payableId: id },
    });
    await syncPayableStatus(id, officeId, tx); // syncPayableStatus precisa aceitar `tx` opcional
    revalidateCase(payable.caseId);
  });
  revalidateFinance();
}

// markManyPayablesPaid/markManyReceivablesPaid: aplicar o MESMO lock por item, dentro do
// loop existente (envolver cada iteração num prisma.$transaction com o SELECT ... FOR UPDATE
// acima) em vez de ler `items` uma vez fora do loop.
```

V2 — Bloquear a gravação desses três campos em updateCase para quem não tem financeAccess (ou exigi-la só quando esses campos vierem alterados em relação ao valor atual), e registrar quem alterou — reaproveitando o padrão já existente em apurarHonorario.

```
// cases.ts – updateCase corrigido (trecho da função, resto inalterado)
export async function updateCase(caseId: string, data: { ...; caseValue?: string; convictionValue?:
string; economicBenefitValue?: string; ... }) {
  const viewer = await getCurrentUser();
  if (!viewer) return { error: "Sessão inválida." };
  const existing = await prisma.case.findFirst({ where: { id: caseId, officeId: viewer.officeId },
select: { caseValue: true, convictionValue: true, economicBenefitValue: true, ... } });
  if (!existing) return { error: "Processo não encontrado." };

  // [SEGURANÇA] [V2]: as 3 bases de cálculo de honorário só mudam de valor com acesso
  // financeiro – evita que edição de cadastro vire canal indireto de fraude em cobrança.
  const novoCaseValue = data.caseValue ? parseFloat(data.caseValue) : null;
  const novoConvictionValue = data.convictionValue ? parseFloat(data.convictionValue) : null;
  const novoEconomicBenefitValue = data.economicBenefitValue ? parseFloat(data.economicBenefitValue) :
null;
  const mudouBaseFinanceira =
    novoCaseValue !== existing.caseValue ||
    novoConvictionValue !== existing.convictionValue ||
    novoEconomicBenefitValue !== existing.economicBenefitValue;
  if (mudouBaseFinanceira && !(viewer.isAdmin || viewer.financeAccess)) {
    return { error: "Alterar valor da causa/condenação/proveito econômico exige acesso ao Financeiro."
};
  }

  await prisma.case.update({ where: { id: caseId }, data: {
    caseValue: novoCaseValue,
    convictionValue: novoConvictionValue,
    economicBenefitValue: novoEconomicBenefitValue,
    ...
  }});
  // [SEGURANÇA] Auditoria mínima – mesma tabela/padrão já usado por AccessAuditLog em outros pontos
  sensíveis.
  if (mudouBaseFinanceira) {
    await prisma.accessAuditLog.create({ data: { userId: viewer.id, officeId: viewer.officeId, action:
"CASE_FINANCIAL_BASE_UPDATE", targetId: caseId } });
  }
  ...
}
```

V3 — Aplicar exatamente o mesmo padrão já usado por Google/Microsoft/Dropbox: buildOAuthState/verifyAndConsumeOAuthState.

```
// lib/btg.ts
import { buildOAuthState } from "@lib/oauthState"; // [SEGURANÇA] [V3]
export function getBtgAuthorizeUrl(state: string): string {
  const params = new URLSearchParams({ client_id: ..., response_type: "code", redirect_uri:
  redirectUri(), scope: "...", prompt: "login", state });
  return `${AUTH_BASE}/oauth2/authorize?${params.toString()}`;
}

// app/api/btg/connect/route.ts
// [SEGURANÇA] [V3]: state agora é um nonce em cookie httpOnly, não o id do usuário –
// fecha a janela de CSRF/injeção de código que as outras 3 integrações OAuth já fecharam.
return NextResponse.redirect(getBtgAuthorizeUrl(buildOAuthState("btg")));

// app/api/btg/callback/route.ts
import { verifyAndConsumeOAuthState } from "@lib/oauthState";
export async function GET(request: NextRequest) {
  const user = await getCurrentUser();
  if (!user?.isPlatformOwner) return NextResponse.redirect(new URL("/painel", request.url));

  const state = request.nextUrl.searchParams.get("state");
  const verified = verifyAndConsumeOAuthState(state); // [SEGURANÇA] [V3]
  if (!verified) return NextResponse.redirect(new URL("/painel-mestre?btg=erro&msg=state_invalid",
  request.url));

  const code = request.nextUrl.searchParams.get("code");
  ... // resto inalterado
}
```

V4 — Trocar para a distribuição corrigida via CDN da própria SheetJS, ou migrar para exceljs.

```
# [SEGURANÇA] [V4]: remove a versão travada do npm, instala a distribuição corrigida
# publicada pela própria SheetJS (nunca chega ao registry do npm – instalação por tarball).
npm uninstall xlsx
npm install https://cdn.sheetjs.com/xlsx-0.20.3/xlsx-0.20.3.tgz

// lib/importers/parse.ts – sem nenhuma outra mudança de código necessária:
// a API pública (XLSX.read/XLSX.utils) é compatível entre 0.18.5 e 0.20.3.
```

V5 — Adicionar um bloco headers() em next.config.mjs com o conjunto mínimo recomendado pela OWASP.

```
// next.config.mjs
/** @type {import('next').NextConfig} */
const nextConfig = {
  poweredByHeader: false, // [SEGURANÇA] [V5b]: para de anunciar "Next.js" no header X-Powered-By
  async headers() {
    return [
      {
        source: "/*:path*",
        headers: [
          // [SEGURANÇA] [V5]: baseline de headers ausente – clickjacking, MIME-sniffing e
          // downgrade para HTTP não tinham nenhuma barreira além do que o Vercel já faz por padrão.
          { key: "X-Content-Type-Options", value: "nosniff" },
          { key: "X-Frame-Options", value: "DENY" },
          { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
          { key: "Strict-Transport-Security", value: "max-age=63072000; includeSubDomains; preload" },
          {
            key: "Content-Security-Policy",
            // Começa em modo Report-Only – o app usa OAuth de 3 provedores + Vercel Blob +
            // fontes/scripts variados; apertar direto para "enforce" arriscaria quebrar algo
            // sem antes ver os relatórios de violação por um tempo.
            value: "default-src 'self'; frame-ancestors 'none'; base-uri 'self';",
          },
        ],
      },
    ];
  },
};
export default nextConfig;
```

V6 — Somar getCurrentUser() + isPlatformOwner como segunda camada (mesmo padrão de setup-painel-mestre), mover o segredo para um header Authorization, e nunca devolver error.message cru ao cliente.

```
export async function GET(request: NextRequest) {
  // [SEGURANÇA] [V6]: segunda camada de defesa – mesmo com o segredo em mãos, exige
  // também uma sessão de platform owner (padrão já usado por setup-painel-mestre).
  const viewer = await getCurrentUser();
  if (!viewer?.isPlatformOwner) {
    return NextResponse.json({ error: "unauthorized" }, { status: 401 });
  }

  // [SEGURANÇA] [V6]: segredo agora via header, não query string (não fica em log/histórico/Referer).
  const secret = request.headers.get("authorization")?.replace("Bearer ", "");
  const expected = process.env.MIGRATION_SECRET;
  if (!expected || secret !== expected) {
    return NextResponse.json({ error: "unauthorized" }, { status: 401 });
  }
  ...
  try {
    const result = await migrarResultadoLegado({ sourceUrl, destDb: prisma, officeSlug, officeName });
    return NextResponse.json(result, { headers: { "Cache-Control": "no-store" } });
  } catch (error) {
    // [SEGURANÇA] [V6]: detalhe completo só no log do servidor – nunca no corpo da resposta,
    // que podia ecoar host/usuário da string de conexão do banco legado.
    console.error("[migrate-legacy] falha:", error);
    return NextResponse.json({ error: "Erro durante a migração. Veja os logs do servidor." }, { status:
500, headers: { "Cache-Control": "no-store" } });
  }
}
```

V7 — Tornar o `updateMany` condicional ao estado atual (`assignedToId: null`) e checar o `count` retornado antes de considerar a delegação bem-sucedida.

```
if (data.publicationId) {
  // [SEGURANÇA] [V7]: compare-and-swap – só "vence" quem chegar primeiro; a segunda
  // chamada concorrente recebe count:0 e pode avisar o usuário em vez de duplicar silenciosamente.
  const { count } = await prisma.publication.updateMany({
    where: { id: data.publicationId, officeId: viewer.officeId, assignedToId: null },
    data: { assignedToId: responsibleIds[0] },
  });
  if (count === 0) {
    return { error: "Esta publicação já foi atribuída a outra pessoa enquanto você preenchia o formulário." };
  }
  await resolvePublicationGroupForOffice(data.publicationId, viewer.officeId);
  ...
}
```

V8 — Adicionar `take` (com paginação de verdade nas duas páginas) e um teto de candidatos nas 4 queries de fallback do `globalSearch`.

```
// app/(app)/processos/page.tsx – adiciona paginação real
const PAGE_SIZE = 50;
const page = Number(searchParams.page) || 1;
prisma.case.findMany({
  where, include: {...}, orderBy: SORTS[sortKey],
  take: PAGE_SIZE, skip: (page - 1) * PAGE_SIZE, // [SEGURANÇA] [V8]
});

// lib/actions/search.ts – teto nas 4 queries de fallback
const caseCandidates = await prisma.case.findMany({ where: { officeId }, select: {...}, take: 500 });
// [SEGURANÇA] [V8]
```

V9 — Substituir por uma biblioteca mantida (`isomorphic-dompurify`) como defesa em profundidade, mesmo sem um exploit confirmado hoje.

```
npm install isomorphic-dompurify

// lib/richText.ts
import DOMPurify from "isomorphic-dompurify";

const ALLOWED_TAGS = ["b", "strong", "i", "em", "u", "ul", "ol", "li", "br", "p", "div", "span"];

// [SEGURANÇA] [V9]: troca regex por um parser HTML real (DOMPurify) – mesma allowlist de
// tags de antes, mas a análise agora é feita sobre uma árvore DOM de verdade, não uma
// sequência de replace() que um caso de borda do parser do navegador pode escapar.
export function sanitizeRichTextHtml(html: string): string {
  if (!html) return "";
  return DOMPurify.sanitize(html, { ALLOWED_TAGS, ALLOWED_ATTR: [] }).trim();
}
```


V10 — Cada uma dessas funções deve derivar `officeId` de `getCurrentUser()` internamente; se precisarem continuar genéricas para uso interno (cron/scripts), mover para um módulo comum sem "use server" em vez de ficarem no mesmo arquivo das Server Actions client-reachable.

```
// lib/actions/attendancePendencias.ts
// [SEGURANÇA] [V10]: officeId deixa de ser parâmetro – deriva da sessão, como toda outra
// Server Action do arquivo já faz.
export async function autoResolvePendenciasForAttachment(attendanceId: string, docType: string):
Promise<void> {
  const viewer = await getCurrentUser();
  if (!viewer) return;
  const kind = DOC_TYPE_TO_PENDENCIA_ENVIAR[docType];
  if (!kind) return;
  try {
    await prisma.atendimentoPendencia.updateMany({
      where: { attendanceId, officeId: viewer.officeId, direction: "ENVIAR", kind, status: "PENDENTE"
    },
    data: { status: "CONCLUIDA", completedAt: new Date() },
  });
} catch { /* best-effort */ }
}
// Chamador (lib/actions/attachments.ts) para de passar officeId:
autoResolvePendenciasForAttachment(attendanceId, docType)

// ensureRecurringFeeReceivables/ensureRecurringExpensePayables/syncReceivableStatus: se o
// uso "todos os escritórios" (cron) é intencional e precisa continuar existindo, mover
// esse caminho para um arquivo SEM "use server" (ex.: lib/cron/recurringFees.ts, chamado só
// pela rota app/api/cron/recurring-fees/route.ts, que já é fail-closed por CRON_SECRET) –
// tirando a função do manifesto de Server Actions client-reachable por completo.
```

V11 — Envolver o caminho de sucesso em `prisma.$transaction` (mesmo padrão já usado no branch IMPROCEDENTE), e checar se já existe uma tarefa de conferência aberta antes de criar outra.

```
// [SEGURANÇA] [V11]: mesmo padrão de transação já usado no branch IMPROCEDENTE, aplicado ao sucesso.
await prisma.$transaction(async (tx) => {
  const pendentes = await tx.receivable.findMany({ where: { status: "A_APURAR", ... } });
  for (const r of pendentes) {
    await tx.receivable.update({ where: { id: r.id }, data: { status: "PENDENTE", amount: valorApurado,
... } });
  }
  const jaExiste = await tx.task.findFirst({ where: { caseId, type: "TAREFA", title: "Conferir trânsito
em julgado", status: { not: "CONCLUIDO" } } });
  if (!jaExiste) await tx.task.create({ data: { title: "Conferir trânsito em julgado", ... } });
});
```

V12 — Restringir `contentType` a um allowlist raster (png/jpeg/webp) explicitamente no `put()`, rejeitando svg+xml.

```
const ALLOWED_IMAGE_TYPES = new Set(["image/png", "image/jpeg", "image/webp"]); // [SEGURANÇA] [V12]
if (!ALLOWED_IMAGE_TYPES.has(file.type)) {
  return NextResponse.json({ error: "Envie uma imagem PNG, JPEG ou WEBP." }, { status: 400 });
}
const blob = await put(`perfil/${user.id}-${Date.now()}-${file.name}`, file, { access: "public",
contentType: file.type });
```

V13 — Restringir a rota a fotos já vinculadas a um BlogPost publicado (a única situação em que a foto já é intencionalmente pública) — qualquer outra Photo passa a exigir sessão.

```
export async function GET(_request: Request, { params }: { params: { id: string } }) {
  const photo = await prisma.photo.findUnique({ where: { id: params.id } });
  if (!photo) return new Response("Foto não encontrada", { status: 404 });

  // [SEGURANÇA] [V13]: só é pública quando já está anexada a um post PUBLICADO – o único
  // caso em que "pública" já era a intenção. Qualquer outra Photo exige sessão do MESMO office.
  const emPostPublicado = await prisma.blogPost.findFirst({ where: { photoId: photo.id, status:
"PUBLICADO" }, select: { id: true } });
  if (!emPostPublicado) {
    const viewer = await getCurrentUser();
    if (!viewer || viewer.officeId !== photo.officeId) {
      return new Response("Foto não encontrada", { status: 404 });
    }
  }
  ... // resto inalterado (busca no Blob e serve os bytes)
}
```

Fase 3 — Testes de segurança (Security TDD)

O repositório não tem framework de teste configurado hoje (sem jest/vitest em package.json). Os testes abaixo usam sintaxe estilo Vitest como referência — para rodar de verdade, instalar vitest e um helper de fixtures de banco de teste antes.

V1

```
// tests/financeiro.race.test.ts (vitest — instalar vitest + @vitest/... antes de rodar)
test("VULN-1: duas baixas concorrentes na mesma conta não duplicam o pagamento", async () => {
  const payable = await criarPayableDeTeste({ amount: 5000 });
  const [r1, r2] = await Promise.allSettled([
    markPayablePaid(payable.id, 5000, "2026-09-05"),
    markPayablePaid(payable.id, 5000, "2026-09-05"),
  ]);
  const sucesso = [r1, r2].filter((r) => r.status === "fulfilled").length;
  expect(sucesso).toBe(1); // só uma das duas deve ter sido aceita
  const pagamentos = await prisma.financePayment.findMany({ where: { payableId: payable.id } });
  const total = pagamentos.reduce((s, p) => s + p.amount, 0);
  expect(total).toBe(5000); // nunca 10000
});
```

V2

```
test("VULN-2: usuário sem financeAccess não consegue alterar valor de condenação do processo", async ()
=> {
  const user = await criarUsuarioDeTeste({ isAdmin: false, financeAccess: false });
  const kase = await criarCaseDeTeste({ convictionValue: 1000 });
  const result = await asUser(user).updateCase(kase.id, { convictionValue: "999999" });
  expect(result.error).toMatch(/Financeiro/);
  const atual = await prisma.case.findUnique({ where: { id: kase.id } });
  expect(atual?.convictionValue).toBe(1000); // não mudou
});
```

V3

```
test("VULN-3: callback do BTG recusa quando o state não bate com o cookie da sessão", async () => {
  const res = await fetch("/api/btg/callback?code=codigo-do-atacante&state=btg:nonce-forjado", {
    headers: { cookie: sessionCookiePlatformOwner }, // sem o cookie lumen-oauth-state correspondente
  });
  expect(res.url).toMatch(/btg=erro/);
  const conn = await prisma.btgConnection.findFirst();
  expect(conn?.accessToken).not.toBe("token-do-atacante");
});
```

V4

```
test("VULN-4: dependência xlsx não está mais na versão vulnerável do npm", () => {
  const pkg = require("../package.json");
  expect(pkg.dependencies.xlsx).not.toMatch(/^(?0\18\./);
});
```

V5

```
test("VULN-5: resposta HTTP inclui os headers de segurança mínimos", async () => {
  const res = await fetch("https://lumen-flax-chi.vercel.app/");
  expect(res.headers.get("x-frame-options")).toBe("DENY");
  expect(res.headers.get("x-content-type-options")).toBe("nosniff");
  expect(res.headers.get("strict-transport-security")).toMatch(/max-age=/);
});
```

V6

```
test("VULN-6: migrate-legacy recusa mesmo com o secret certo, sem sessão de platform owner", async () => {
  const res = await fetch("/api/admin/migrate-legacy?officeSlug=x", {
    headers: { authorization: `Bearer ${process.env.MIGRATION_SECRET}` }, // sem cookie de sessão
  });
  expect(res.status).toBe(401);
});
```

V7

```
test("VULN-7: duas delegações concorrentes na mesma publicação não criam duas tarefas", async () => {
  const pub = await criarPublicacaoDeTeste({ assignedToId: null });
  const [r1, r2] = await Promise.allSettled([
    delegateTask({ responsibleIds: ["user-a"], publicationId: pub.id, ... }),
    delegateTask({ responsibleIds: ["user-b"], publicationId: pub.id, ... }),
  ]);
  const semErro = [r1, r2].filter((r) => r.status === "fulfilled" && !r.value.error);
  expect(semErro.length).toBe(1);
});
```

V8

```
test("VULN-8: listagem de processos nunca devolve mais que PAGE_SIZE registros", async () => {
  await criarNCasesDeTeste(200);
  const html = await renderProcessosPage({ page: "1" });
  expect(contarLinhasDaTabela(html)).toBeLessThanOrEqual(50);
});
```

V9

```
test("VULN-9: sanitizeRichTextHtml remove onerror/onload mesmo em tags não bloqueadas por nome", () => {
  expect(sanitizeRichTextHtml('<p onclick="alert(1)">x</p>')).toBe("<p>x</p>");
  expect(sanitizeRichTextHtml('<svg><script>alert(1)</script></svg>')).toBe("");
});
```

V10

```
test("VULN-10: autoResolvePendenciasForAttachment não aceita mais officeId de outro tenant", () => {
  // typecheck: a assinatura não tem mais o 3º parâmetro – o teste é de compilação
  // @ts-expect-error officeId não é mais um parâmetro válido
  autoResolvePendenciasForAttachment("att-1", "CONTRATO", "office-de-outro-tenant");
});
```

V11

```
test("VULN-11: duas apurações concorrentes não criam duas tarefas de conferência", async () => {
  await Promise.allSettled([apurarHonorario(caseId, base, valor), apurarHonorario(caseId, base, valor)]);
  const tarefas = await prisma.task.count({ where: { caseId, title: "Conferir trânsito em julgado" } });
  expect(tarefas).toBe(1);
});
```

V12

```
test("VULN-12: upload de foto de perfil rejeita SVG", async () => {
  const svg = new File(["<svg onload=alert(1)></svg>"], "x.svg", { type: "image/svg+xml" });
  const res = await uploadPerfilFoto(svg);
  expect(res.status).toBe(400);
});
```

V13

```
test("VULN-13: foto não publicada de outro escritório não é acessível sem sessão do mesmo tenant",
  async () => {
    const photo = await criarPhotoDeTeste({ officeId: "office-b" }); // não publicada
    const res = await fetch(`/api/photos/file/${photo.id}`); // sem cookie de sessão
    expect(res.status).toBe(404);
  });
```

Recomendações priorizadas

Prior.	Recomendação	Achado
P1	Corrigir a corrida de pagamento duplicado (V1) — é o achado de maior impacto financeiro real, some direto no Livro Caixa/DRE/Fluxo de Caixa.	V1
P1	Bloquear updateCase de alterar valor da causa/condenação/proveito econômico sem financeAccess (V2).	V2
P1	Aplicar verifyAndConsumeOAuthState ao fluxo BTG, igual às outras 3 integrações OAuth (V3).	V3
P1	Trocar a dependência xlsx pela distribuição corrigida da SheetJS ou migrar para exceljs (V4).	V4
P2	Configurar os headers de segurança em next.config.mjs (V5) e fechar a lacuna de autenticação de migrate-legacy (V6).	V5, V6
P2	Aplicar compare-and-swap em Publication.assignedToId (V7) e adicionar paginação real a Processos/Clientes/globalSearch (V8).	V7, V8
P3	Substituir o sanitizador regex por isomorphic-dompurify (V9); derivar officeld da sessão nas 4 funções apontadas em V10 (ou movê-las para fora do arquivo "use server").	V9, V10
P3	Envolver apurarHonorario em transação (V11), restringir upload de foto a PNG/JPEG/WEBP (V12), e restringir /api/photos/file a fotos já publicadas ou do mesmo escritório (V13).	V11, V12, V13